

2025 年全国大学测绘学科 创新创业智能大赛 测绘程序设计比赛

报告文档

——点云去噪分析

目录

一、 程序优化性说明.....	3
1. 用户交互界面说明.....	3
2. 程序运行过程说明.....	3
2.1 导入数据.....	3
2.2 执行计算.....	4
2.3 保存结果.....	5
3. 程序运行结果说明.....	6
二、 程序规范性说明.....	7
1. 程序功能与结构设计说明.....	7
1.1 程序功能.....	7
1.2 结构设计说明.....	7
2. 核心算法源码.....	7
2.1 关键函数汇总.....	7
2.2 关键源码展示.....	8
三、 程序完整性源代码.....	13
1. Form.cs 完整源代码.....	13
2. Point 类完整源代码.....	18
3. Algo 完整源代码.....	19

一、程序优化性说明

1. 用户交互界面说明

程序采用基本的 WinForm 交互界面，包括菜单、工具栏、输出报告等功能。功能正确、可正常运行，布局合理、直观美观、人性化。

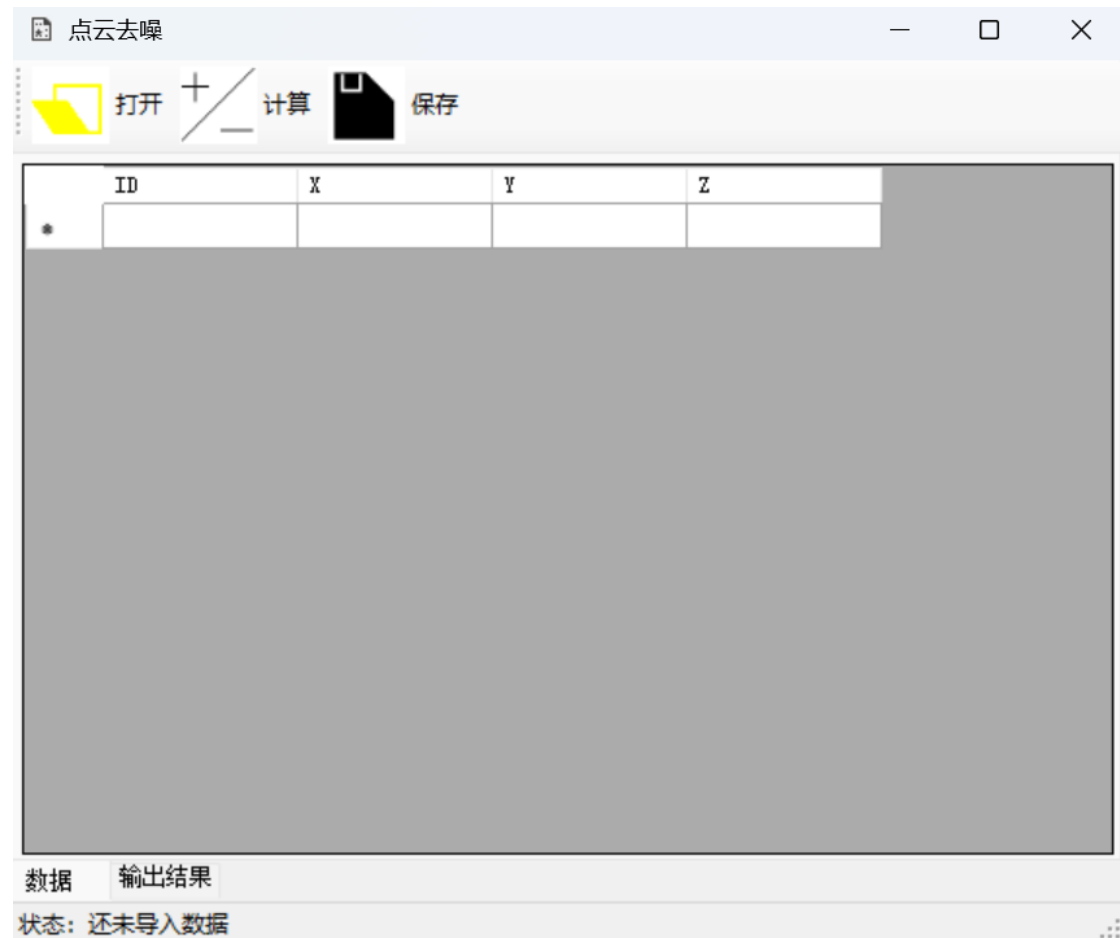


图 1 程序基本界面

2. 程序运行过程说明

2.1 导入数据

用户点击程序界面的“打开”按钮，即可选择文本数据进行导入，导入成功后会有消息提示。

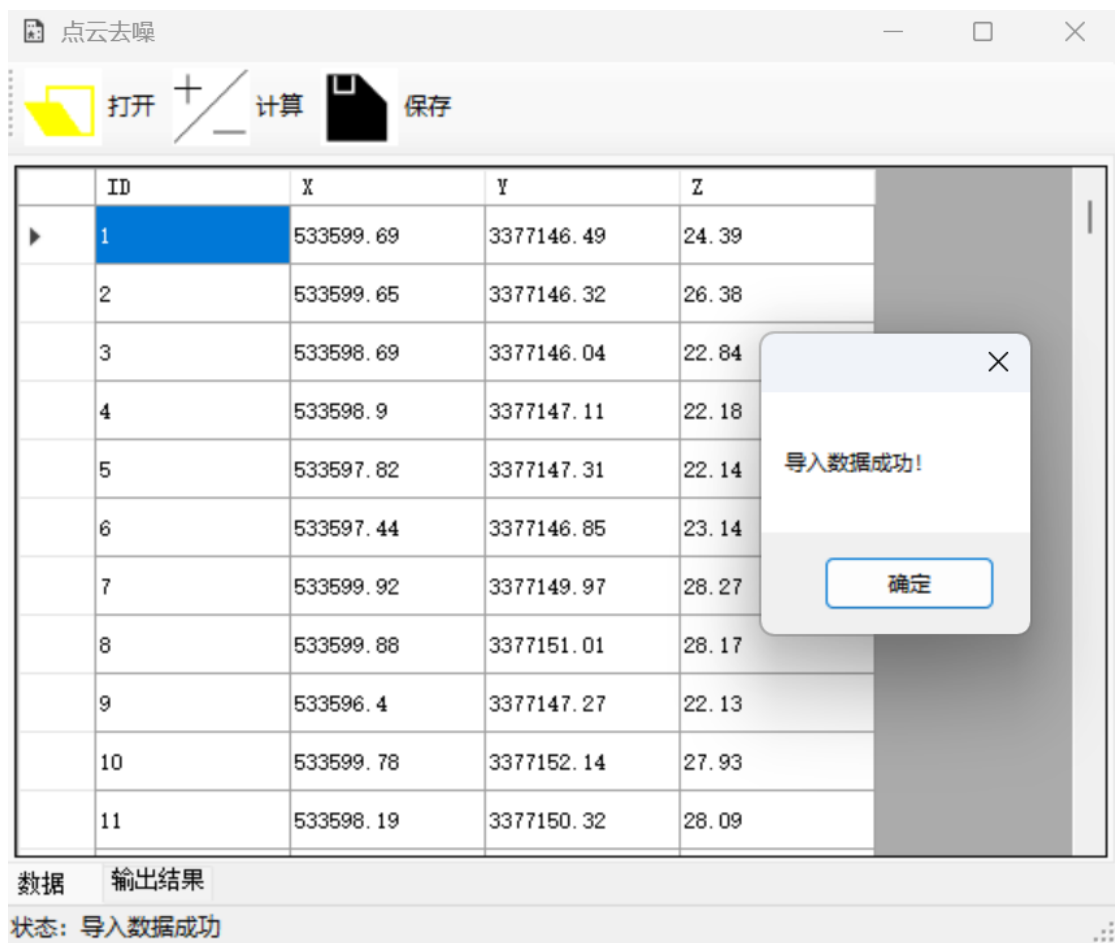


图 2 导入数据

2.2 执行计算

用户点击程序界面的“计算”按钮，即可执行点云去噪计算，计算成功后，程序自动跳转到输出结果界面展示结果。

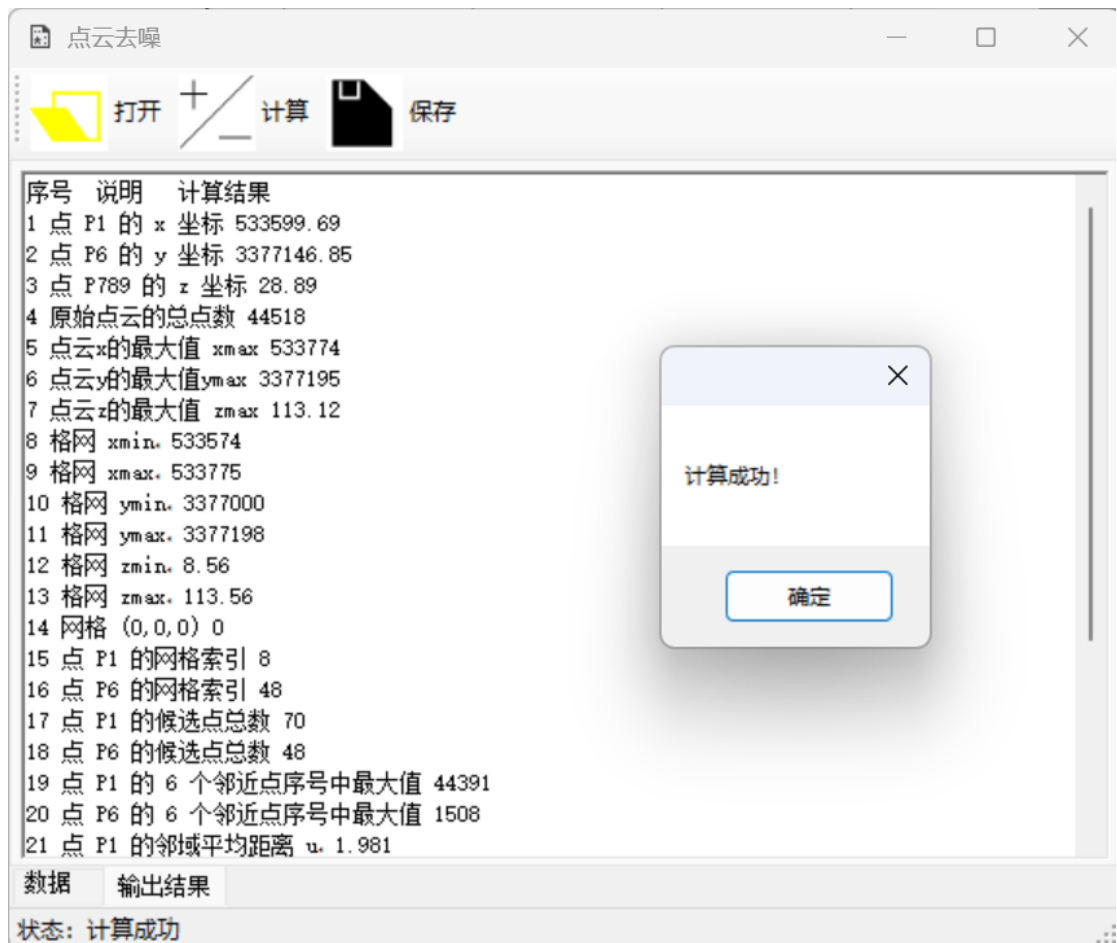


图 3 执行计算

2.3 保存结果

用户点击程序界面的“保存”按钮，即可选择保存路径，然后保存最终结果。

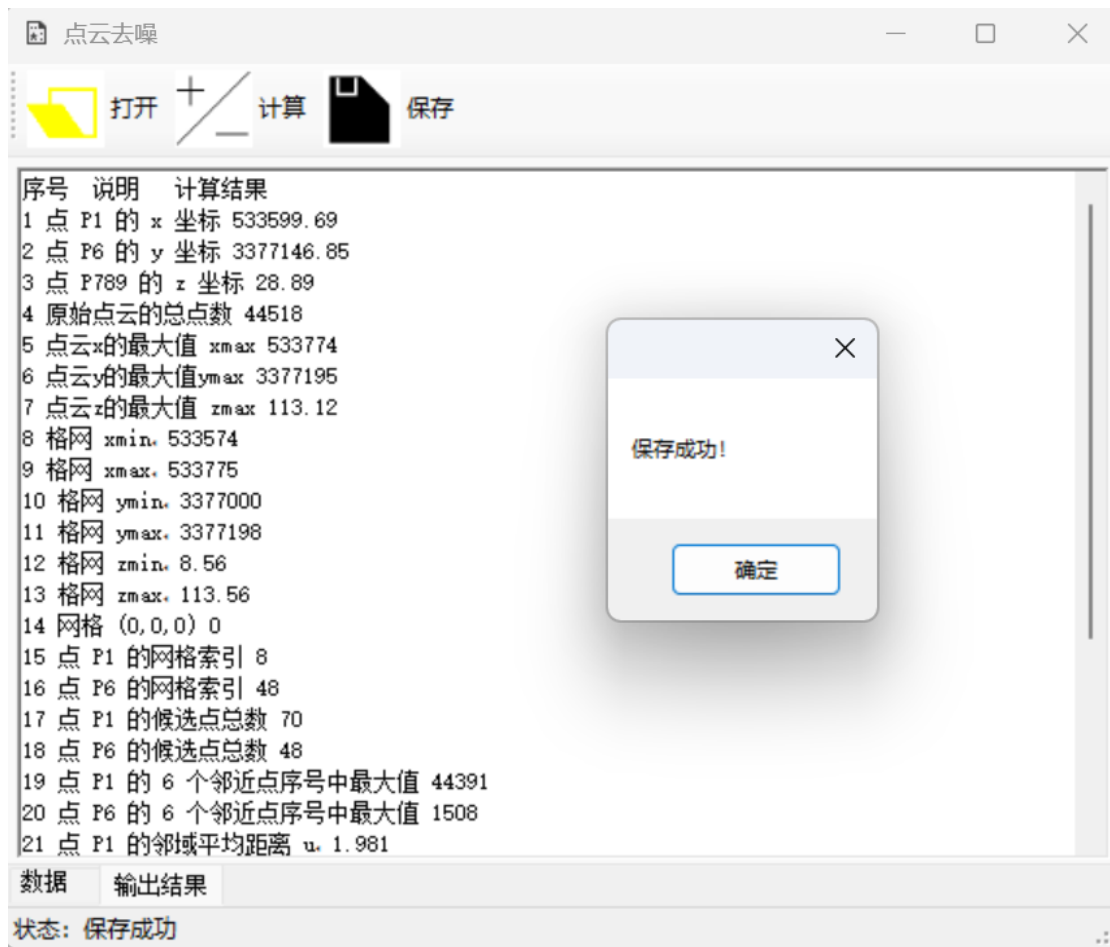


图 4 保存输出

3. 程序运行结果说明

导入数据成功后，执行计算成功后，程序会生成计算结果报告，包括 30 项结果输出。

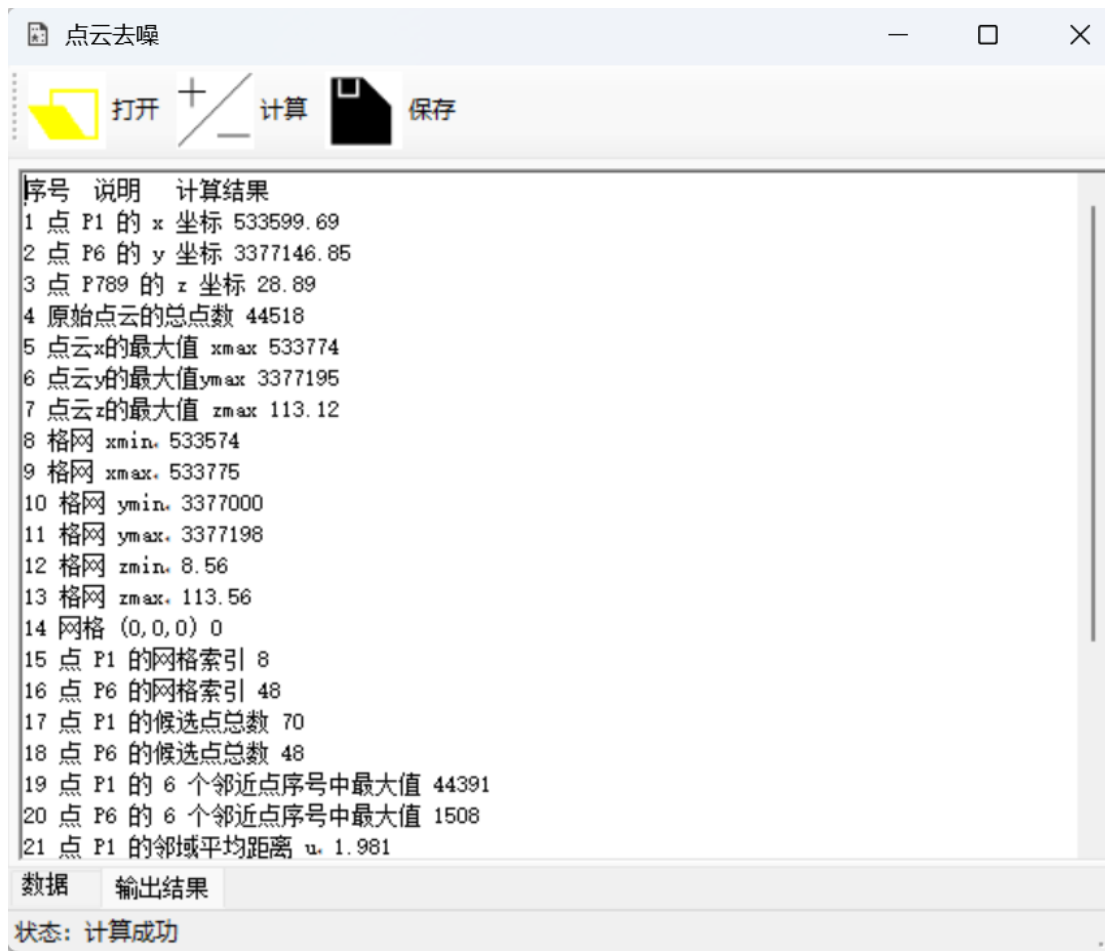


图 5 程序运行结果

二、程序规范性说明

1. 程序功能与结构设计说明

1.1 程序功能

程序主要基于统计滤波的点云去噪算法，实现流程包括数据读取、格网划分、k 邻近搜索、统计特征计算及噪声判断。最终输出结果。

1.2 结构设计说明

程序主要结构分为 3 大功能模块，一是数据导入与导出模块，实现数据的存储与结果的输出。二是点云 Point 基础类模块，实现点云基本参数的保存，包括每个点的 ID，所在网格单元以及统计特征等信息。

2. 核心算法源码

2.1 关键函数汇总

序号	类名	函数名	输入参数	输出参数	主要功能
1	Point	GetDistance	point1	void	点与点之间欧式距离计算

2	Algo	GetGrid	无	void	计算格网基本参数
3	Algo	Points2Grid	无	void	点云网格分配
4	Algo	SearchK	无	void	K 邻近点搜索
5	Algo	RemovePoints	无	void	标记噪声点

2.2 关键源码展示

2.2.1 Point 类

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Text.RegularExpressions;

namespace Linda
{
    class Point
    {
        // 每个点的数据结构
        public int ID; // 编号 ID
        public double X; // X 坐标
        public double Y; // Y 坐标
        public double Z; // Z 坐标

        // 点所在点云的网格单元;
        public int i, j, k;

        // 与中心点的距离, k 邻近搜索需要用到
        public double di;

        // 统计特征
        public double ui; //平均距离
        public double fanchai; //标准差

        //是否为噪声点,ture 表示是噪声点
        public bool IS_bad = false;

        // Regx 分割输入的每一行
```



```

    public Point(string line, int id)
    {
        var data = Regex.Split(line, @"\s"); //以空格分隔
        ID = id;
        X = Convert.ToDouble(data[0]);
        Y = Convert.ToDouble(data[1]);
        Z = Convert.ToDouble(data[2]);
    }

    public void GetDistance(Point point)
    {
        di = Math.Sqrt(Math.Pow(X - point.X, 2) + Math.Pow(Y
- point.Y, 2) + Math.Pow(Z - point.Z, 2));
    }
}

```

2.2.2 GetGrid

```

// 第一步，计算格网基本参数
public void GetGrid()
{
    // 首先确定点云 xyz 最大最小值
    result.xmax = points.OrderByDescending(p =>
p.X).FirstOrDefault().X;
    result.ymax = points.OrderByDescending(p =>
p.Y).FirstOrDefault().Y;
    result.zmax = points.OrderByDescending(p =>
p.Z).FirstOrDefault().Z;

    result.xmin = points.OrderBy(p =>
p.X).FirstOrDefault().X;
    result.ymin = points.OrderBy(p =>
p.Y).FirstOrDefault().Y;
    result.zmin = points.OrderBy(p =>
p.Z).FirstOrDefault().Z;

    // 然后计算点云空间格网最大值

```

```

        result.xmax1 = Math.Floor((result.xmax -
result.xmin) / 3.0 + 1) * 3.0 + result.xmin;
        result.ymax1 = Math.Floor((result.ymax -
result.ymin) / 3.0 + 1) * 3.0 + result.ymin;
        result.zmax1 = Math.Floor((result.zmax -
result.zmin) / 3.0 + 1) * 3.0 + result.zmin;
    }

```

2.2.3 Points2Grid

```

// 第二步，进行点云分配
public void Points2Grid()
{
    foreach(var p in points)
    {
        int i = (int)Math.Floor((p.X - result.xmin) /
3.0);
        int j = (int)Math.Floor((p.Y - result.ymin) /
3.0);
        int k = (int)Math.Floor((p.Z - result.zmin) /
3.0);

        var key = Tuple.Create(i, j, k);
        // 判断 grid 有没有对应的 key
        if (!grid.ContainsKey(key))
        {
            // 如果没有，即第一次创建，那么要 new list
            grid[key] = new List<Point>();
        }

        // 将点加入对应的网格空间
        grid[key].Add(p);

        p.i = i;
        p.j = j;
        p.k = k;
    }
    // 程序正确性，网格(0,0,0)内点的个数
    var key_000 = Tuple.Create(0, 0, 0);

```

```

        if (grid.ContainsKey(key_000))
        {
            result.count_grid000 = grid[key_000].Count;
        }
        else
        {
            result.count_grid000 = 0;
        }
    }
}

```

2.2.4 SearchK

```

// 第三步, k 邻近点搜索, k = 6
public void SearchK()
{
    // 相邻网格, 27 个, 边界网格除外
    foreach(var p in points)
    {
        List<Point> condidate_points = new
List<Point>(); // 候选点
        List<Point> k_points = new List<Point>(); // 最
最终选择最近的 6 个点
        for(int i = -1; i <= 1; i++)
        {
            for (int j = -1; j <= 1; j++)
            {
                for (int k = -1; k <= 1; k++)
                {
                    var key = Tuple.Create(p.i + i, p.j +
j, p.k + k);
                    //首先判断有无 key, 因为边界网格不一定是
27 个

                    if (grid.ContainsKey(key))
                    {
                        condidate_points.AddRange(grid[key]);
                    }
                }
            }
        }
    }
}

```

```

        }
    }
    // 选择最近的 6 个点
    // 首先计算距离
    foreach(var condidate_p in condidate_points)
    {
        condidate_p.GetDistance(p);
    }
    // 排除自身
    k_points = condidate_points.Where(point =>
point.di > 1e-5).ToList();
    // 取最近的前 6 个
    k_points = k_points.OrderBy(point =>
point.di).Take(6).ToList();

    // 如果为 0，说明这个点及其偏远
    if(k_points.Count == 0)
    {

    }

    // 统计特征
    // 平均距离
    double ui = k_points.Average(point => point.di);
    p.ui = ui;
    // 方差
    double fanchai = k_points.Average(point =>
(point.di - ui) * (point.di - ui));
    fanchai = Math.Sqrt(fanchai);
    p.fanchai = fanchai;
    }
}

```

2.2.5 RemovePoints

```

// 第四步，去除噪声点
public void RemovePoints()
{
    // 首先计算所有点的平均均值和标准差

```

```

        double u_global = points.Average(p => p.ui);
        double fancha_global = points.Average(p =>
p.fanchai);

        // 标记噪声点
        foreach(var p in points)
        {
            if(p.ui > u_global + 2.0 * fancha_global)
            {
                // 标记为噪声点
                p.IS_bad = true;
            }
        }

        // 去噪后的点云总数
        int count = 0;
        foreach(var p in points)
        {
            if (!p.IS_bad)
            {
                count++;
            }
        }
    }
}

```

三、程序完整性源代码

1. Form.cs 完整源代码

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using System.IO;

```

```

namespace Linda
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }
        List<Point> points = new List<Point>(); //原始点云数据

        private void toolStripButton1_Click(object sender,
EventArgs e)
        {
            try
            {
                //导入数据
                OpenFileDialog openFileDialog1 = new
OpenFileDialog
            {
                FileName = "导入数据",
                Filter = "点云数据|*.txt",
                Title = "请选择要处理的数据"
            };
                if(openFileDialog1.ShowDialog() ==
DialogResult.OK)
                {
                    var reader = new
StreamReader(openFileDialog1.FileName);

                    points.Clear(); //清空原有数据
                    int temp_ID = 0; //临时变量，用于给点进行 ID 编
号

                    while (!reader.EndOfStream)
                    {

                        var line = reader.ReadLine();
                        // 防止为空

```

```

        if(line.Length > 0)
        {
            temp_ID++;
            var point = new Point(line, temp_ID);
            points.Add(point);
        }
    }
    reader.Close();

    // dataGridview 更新数据
    foreach(var p in points)
    {
        dataGridview1.Rows.Add(p.ID, p.X, p.Y,
p.Z);
    }
    // 提示内容
    toolStripStatusLabel1.Text = "状态: 导入数据成
功";

    tabControl1.SelectedIndex = 0;
    MessageBox.Show("导入数据成功!");
}
}
catch (Exception ex)
{
    throw ex;
}

private void toolStripButton2_Click(object sender,
EventArgs e)
{
    // 执行计算
    try
    {
        // 先判断有没有导入数据
        if(points.Count == 0)
        {

```

```

        MessageBox.Show("请先导入数据！");
        return;
    }

    Algo algo = new Algo(points);

    richTextBox1.Text =
    $"序号 说明 计算结果
    1 点 P1 的 x 坐标 533599.69
    2 点 P6 的 y 坐标 3377146.85
    3 点 P789 的 z 坐标 28.89
    4 原始点云的总点数 44518
    5 点云 x 的最大值 xmax 533774
    6 点云 y 的最大值 ymax 3377195
    7 点云 z 的最大值 zmax 113.12
    8 格网 xmin1 533574
    9 格网 xmax1 533775
    10 格网 ymin1 3377000
    11 格网 ymax1 3377198
    12 格网 zmin1 8.56
    13 格网 zmax1 113.56
    14 网格 (0,0,0) 0
    15 点 P1 的网格索引 8
    16 点 P6 的网格索引 48
    17 点 P1 的候选点总数 70
    18 点 P6 的候选点总数 48
    19 点 P1 的 6 个邻近点序号中最大值 44391
    20 点 P6 的 6 个邻近点序号中最大值 1508
    21 点 P1 的邻域平均距离  $u_1$  1.981
    22 点 P1 的邻域距离标准差  $\sigma_1$  0.329
    23 点 P6 的邻域平均距离  $u_6$  1.411
    24 点 P6 的邻域距离标准差  $\sigma_6$  0.186
    25 全局平均距离均值  $\mu$  1.668
    26 全局距离标准差  $\sigma$  0.339
    27 点 P1 是否为噪声点 1
    28 点 P6 是否为噪声点 0
    29 去噪后保留的点云总数 42555
    ";

```



```

        // 提示内容
        toolStripStatusLabel1.Text = "状态: 计算成功";
        tabControl1.SelectedIndex = 1;
        MessageBox.Show("计算成功! ");
    }
    catch (Exception ex)
    {
        throw ex;
    }
}

private void toolStripButton3_Click(object sender,
EventArgs e)
{
    try
    {
        // 先判断有没有导入数据
        if (points.Count == 0)
        {
            MessageBox.Show("请先导入数据! ");
            return;
        }

        // 保存结果
        SaveFileDialog saveFileDialog1 = new
SaveFileDialog
        {
            FileName = "输出报告",
            Filter = "result|*.txt",
            Title = "请选择要保存的位置"
        };
        if(saveFileDialog1.ShowDialog() ==
DialogResult.OK)
        {

```

```

        var writer = new
StreamWriter(saveFileDialog1.FileName);

        writer.Write(richTextBox1.Text);

        writer.Close();

        // 提示内容
        toolStripStatusLabel1.Text = "状态: 保存成功";
        tabControl1.SelectedIndex = 1;
        MessageBox.Show("保存成功!");
    }
}
catch (Exception ex)
{
    throw ex;
}
}
}
}

```

2. Point 类完整源代码

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Text.RegularExpressions;

namespace Linda
{
    class Point
    {
        // 每个点的数据结构
        public int ID; // 编号 ID
        public double X; // X 坐标
        public double Y; // Y 坐标
    }
}

```

```

public double Z; // Z坐标

// 点所在点云的网格单元;
public int i, j, k;

// 与中心点的距离, k 邻近搜索需要用到
public double di;

// 统计特征
public double ui; //平均距离
public double fanchai; //标准差

//是否为噪声点,ture 表示是噪声点
public bool IS_bad = false;

// Regx 分割输入的每一行
public Point(string line, int id)
{
    var data = Regex.Split(line, @"\s"); //以空格分隔
    ID = id;
    X = Convert.ToDouble(data[0]);
    Y = Convert.ToDouble(data[1]);
    Z = Convert.ToDouble(data[2]);
}

public void GetDistance(Point point)
{
    di = Math.Sqrt(Math.Pow(X - point.X, 2) + Math.Pow(Y
- point.Y, 2) + Math.Pow(Z - point.Z, 2));
}
}
}

```

3. Algo 完整源代码

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

```

```

using System.Threading.Tasks;

namespace Linda
{
    class Algo
    {
        // 主算法实现
        // 点云数据
        public List<Point> points = new List<Point>();
        // 划分的格网,使用 Dictionary 作为基本结构
        public Dictionary<Tuple<int, int, int>, List<Point>>>
grid = new Dictionary<Tuple<int, int, int>, List<Point>>>();

        // 储存最终的结果
        public Result result = new Result();

        public Algo(List<Point> origin_points)
        {
            // 清除原有数据,防止重复点击
            points.Clear();
            grid.Clear();

            points = origin_points;

            // 第一步,计算格网基本参数
            GetGrid();

            // 第二步,进行点云分配
            Points2Grid();

            // 第三步,k 邻近点搜索, k = 6
            SearchK();

            // 第四步,去除噪声点
            RemovePoints();
        }
    }
}

```

```

// 第一步，计算格网基本参数
public void GetGrid()
{
    // 首先确定点云 xyz 最大最小值
    result.xmax = points.OrderByDescending(p =>
p.X).FirstOrDefault().X;
    result.ymax = points.OrderByDescending(p =>
p.Y).FirstOrDefault().Y;
    result.zmax = points.OrderByDescending(p =>
p.Z).FirstOrDefault().Z;

    result.xmin = points.OrderBy(p =>
p.X).FirstOrDefault().X;
    result.ymin = points.OrderBy(p =>
p.Y).FirstOrDefault().Y;
    result.zmin = points.OrderBy(p =>
p.Z).FirstOrDefault().Z;

    // 然后计算点云空间格网最大值
    result.xmax1 = Math.Floor((result.xmax -
result.xmin) / 3.0 + 1) * 3.0 + result.xmin;
    result.ymax1 = Math.Floor((result.ymax -
result.ymin) / 3.0 + 1) * 3.0 + result.ymin;
    result.zmax1 = Math.Floor((result.zmax -
result.zmin) / 3.0 + 1) * 3.0 + result.zmin;
}

// 第二步，进行点云分配
public void Points2Grid()
{
    foreach(var p in points)
    {
        int i = (int)Math.Floor((p.X - result.xmin) /
3.0);
        int j = (int)Math.Floor((p.Y - result.ymin) /
3.0);
        int k = (int)Math.Floor((p.Z - result.zmin) /
3.0);
    }
}

```

```

        var key = Tuple.Create(i, j, k);
        // 判断 grid 有没有对应的 key
        if (!grid.ContainsKey(key))
        {
            // 如果没有，即第一次创建，那么要 new list
            grid[key] = new List<Point>();
        }

        // 将点加入对应的网格空间
        grid[key].Add(p);

        p.i = i;
        p.j = j;
        p.k = k;
    }
    // 程序正确性，网格(0,0,0)内点的个数
    var key_000 = Tuple.Create(0, 0, 0);
    if (grid.ContainsKey(key_000))
    {
        result.count_grid000 = grid[key_000].Count;
    }
    else
    {
        result.count_grid000 = 0;
    }
}

// 第三步，k 邻近点搜索，k = 6
public void SearchK()
{
    // 相邻网格，27 个，边界网格除外
    foreach(var p in points)
    {
        List<Point> condidate_points = new
List<Point>(); // 候选点
    }
}

```

```

        List<Point> k_points = new List<Point>(); // 最终选择最近的 6 个点
        for(int i = -1; i <= 1; i++)
        {
            for (int j = -1; j <= 1; j++)
            {
                for (int k = -1; k <= 1; k++)
                {
                    var key = Tuple.Create(p.i + i, p.j + j, p.k + k);
                    //首先判断有无 key，因为边界网格不一定是 27 个

                    if (grid.ContainsKey(key))
                    {
                        condidate_points.AddRange(grid[key]);
                    }
                }
            }
        }
        // 选择最近的 6 个点
        // 首先计算距离
        foreach(var condidate_p in condidate_points)
        {
            condidate_p.GetDistance(p);
        }
        // 排除自身
        k_points = condidate_points.Where(point => point.di > 1e-5).ToList();
        // 取最近的前 6 个
        k_points = k_points.OrderBy(point => point.di).Take(6).ToList();

        // 如果为 0，说明这个点及其偏远
        if(k_points.Count == 0)
        {
        }
    }

```

```

        // 统计特征
        // 平均距离
        double ui = k_points.Average(point => point.di);
        p.ui = ui;
        // 方差
        double fanchai = k_points.Average(point =>
(point.di - ui) * (point.di - ui));
        fanchai = Math.Sqrt(fanchai);
        p.fanchai = fanchai;
    }
}

// 第四步，去除噪声点
public void RemovePoints()
{
    // 首先计算所有点的平均均值和标准差
    double u_global = points.Average(p => p.ui);
    double fancha_global = points.Average(p =>
p.fanchai);

    // 标记噪声点
    foreach(var p in points)
    {
        if(p.ui > u_global + 2.0 * fancha_global)
        {
            // 标记为噪声点
            p.IS_bad = true;
        }
    }

    // 去噪后的点云总数
    int count = 0;
    foreach(var p in points)
    {
        if (!p.IS_bad)
        {
            count++;
        }
    }
}

```



```
    }  
    }  
}  
  
class Result  
{  
    // 储存最终的程序正确性结果  
    public double xmax, ymax, zmax, xmin, ymin, zmin;  
    public double xmax1, ymax1, zmax1;  
    public int count_grid000;  
}  
}
```